deti universidade de aveiro departamento de eletrónica, telecomunicações e informática

HW1: Mid-term assignment

I. Oliveira, v2026-10-10

Assignment objectives

Plan, implement and explain a **test automation strategy** for a small full-stack application. The assessed result is your QA strategy and the evidence that it works.
By the end of the assignment, you should be able to show that you can, autonomously:

- elicit and specify requirements for a simple problem, covering business logic and non-functional qualities, in a form that can be tested;
- choose the right test level and the right test double for each requirement, and justify the choice;
- automate unit, integration, functional, acceptance and non-functional tests, from the build;
- read static-analysis and coverage results critically, and act on them.

This is an **individual** assignment. Generative AI may be used to write code, under the conditions in section "Use of generative AI".

Project scope

Domain context

TuttiBox[1] runs a network of smart parcel lockers. Each **locker station** (e.g. at a supermarket or a train station) has a set of **compartments** in three sizes (S, M, L). **Partners** (mainly online shops) send parcels to a locker station chosen by the customer; a **courier** deposits the parcel in a compartment; the **recipient** (consumer) collects it with a pickup in a secure way. An **operator** supervises the network (live monitor the network operational conditions). Sometimes, the lockers are used for returning items or to send items by individual users.
Services such as Vinted Go or InPost are good references for how real things work; try to discover domain requirements from the existing platforms.

The scope is deliberately **limited and closed**. Anything not listed is out of scope (see also "Out of scope" section).





[1] Fictional name; replace with your own.

Use cases (required core)

ID	Actor	Channel	Use case
UC1	Recipient	Web	**Find a locker.** Enter an address; the system geolocates it (external API, see below) and shows the nearest stations, with how many free compartments each one has per size.
UC2	Partner	REST API	**Register a parcel** for a station: size, recipient contact. The system reserves a compartment and returns a tracking code.
UC3	Courier	Locker terminal	**Drop off.** Enter the tracking code; the reserved compartment opens; the courier closes the door; the parcel becomes ready for pickup, and a pickup code is issued to the recipient.
UC4	Recipient	Locker terminal	**Pick up.** Enter the pickup code; the compartment opens; the parcel is delivered and the compartment is freed.
UC5	Recipient, Partner	Web and REST API	**Track** a parcel with its tracking code: current state and the timestamped history of transitions.
UC6	Operator	Web	**Supervise** a station: list parcels by state, see expired parcels, register their removal, put a faulty compartment out of service.

The **locker terminal** is to be represented by a web interface that plays the role of the touchscreen on the station; how you simulate it is your choice. It does not talk to hardware: the backend does, through the port described in "Decoupling the locker hardware".
Staging data: locker stations and compartments are seed data (a few stations with real coordinates in one city, with several compartments each, mixed sizes). There is **no user registration** and no authentication: just map each actor to its own pages/endpoints. No need for extensive CRUD.

Parcel lifecycle

The lifecycle of a parcel (package in transit) is the core of the business logic. Consider, at least, these minimum states and transitions in the underlying state machine:

From state…	Event (who)	… new state
	Parcel registered, compartment reserved (Partner)	CREATED
CREATED	Parcel deposited and door confirmed closed (Courier)	READY_FOR_PICKUP
CREATED	Cancelled by the partner, or drop-off deadline passed (Partner, System)	CANCELLED
READY_FOR_PICKUP	Valid pickup code entered and compartment opened (Recipient)	PICKED_UP
READY_FOR_PICKUP	Pickup deadline passed (System)	EXPIRED
EXPIRED	Parcel removed from the compartment (Operator)	RETURNED

Ensure every transition is recorded with a timestamp and the actor. Any event that is not valid in the current state must be rejected, and the rejection must be explicit (e.g. an HTTP status and an an propriate error body).
This table is the **minimum**, and the state names are indicative. Part of your work is to **complete the specification**: what else happens that should be guarded? You may refine the model (e.g. add states) if your rules need it.[2]

[2] How you represent the state machine in code is your decision: a dedicated state-machine library is not expected (it is not forbidden either; if you use one, justify it).

Business rules to specify

Define concrete values for the following parameters and make them configurable:

- **Compartment allocation:** a parcel only goes into a compartment of a compatible size; one parcel per compartment; what happens when the station is full; two partners must never get the same last compartment.
- **Deadlines:** time allowed for drop-off after registration; time allowed for pickup after drop-off.
- **Pickup code:** format, single use, validity, and protection against guessing.
- **Hardware failures:** what the system does, for the parcel and for the compartment, when the hardware does not behave as expected.
- Time-based rules must be testable without waiting/sleeping.

Decoupling the locker hardware

The backend must not depend on real hardware for QA. Define a **port** (an interface owned by your code) with the minimum operations the business logic needs. You should implement:

- an **in-memory fake** implementation of that port, with state, that can be configured to **inject failures** that matter for your rules;
- tests that use that fake to check the failure rules you specified;
- a (demo) configuration in which the fake behaves as a working locker, so that the terminal page can be used end to end with success.

Real devices (cyber-physical), communication protocols (MQTT…) and hardware simulators with their own interface are **out of scope**. A simple fake is what is asked.

External API: geolocation

UC1 requires converting an address into coordinates (**geocoding**), so that stations can be ranked by distance. You should choose a geocoding provider:

- available as a service (provides a REST API);
- free, with a documented usage policy that allows your use (e.g. OpenStreetMap Nominatim, GeoAPI.pt, or other);
- the behavior when the provider is unavailable or returns nothing is specified and tested;
- **tests in the default build do not call the real API.** Load tests do not call the real provider.

The distance calculation and the ranking of stations belong to your code and should be unit-tested.

Out of scope

Do not spend time on: user accounts and authentication; payments; real SMS/e-mail (instead, send notifications through a port and log them); simulating locker hardware; a mobile app; return-to-sender logistics; several cities or companies. Maps are optional.

System Architecture

Design decisions left to you. The following design decisions are deliberately not prescribed. Each one affects how you test, so the report must state what you decided and why, and the oral presentation may start from any of them:

- how the locker terminal is simulated (the interaction should be supported by a simple web page)

- how the parcel lifecycle is represented in code and where its rules live;
- how time is made controllable in tests;
- the operations of the hardware port, and whether the interaction is synchronous or event-driven;
- how compartment allocation stays correct under concurrent requests;
- how pickup codes are generated, stored and checked;
- how the system behaves when the geocoding provider is slow, unavailable or over its usage limits;
- the design of your REST API (resources, status codes, error format).

Technology stack

- Backend: **Spring Boot 4.x**, Java 21, Maven or Gradle.
- Persistence: a (*dockerized*) relational database (PostgreSQL recommended). Integration tests run against the same database engine as the application (Testcontainers).
- Frontend (Web layer): any technology you like. Keep it simple: robust backend matters more than a rich frontend.
- Logging: the application must produce a useful log of events (at least: every state transition; every call to the hardware port; every access to the external API).

Use of generative AI

You may use AI assistants to write production and test code. Treat generated tests as drafts asn audit them. What you may not delegate is the **QA strategy**: deciding what to test, at which level, with which test double, and which oracle each test uses. Those decisions are yours, are documented in the report, and will be defended in the oral presentation.

- Declare in the report how you used AI (for what, and which parts you chose not to delegate).
- You should audit the code obtained from AI. You are the author of the work you submit.
- In the oral presentation you may be asked, for example, to explain why a transition is tested at a given level. It is better to have a smaller project that you understand and can explain than an extensive project you cannot justify.

Extras (optional)

Extras only count when the core is complete. Suggested extensions to the core assignment:

- Continuous integration (e.g. GitHub Actions) that runs the build and enforces the Quality Gate.
- An operator dashboard with the live occupancy of each station.
- Mobile app as an additional contact point (especially for the Courier).

Submission

The submission consists of:

1. The **code project**, committed to your TQS-Portfolio personal Git repository, under the folder `/HW1-<my student number>`, (e.g.: HW1-123456) with a Readme.md. The Readme.md should state import assumptions and convey instructions to build the project (by others).
2. A **technical report** following the provided report template (place it in subfolder `/docs)`.
3. A **short video** (screencast, up to 2 minutes) demonstrating the core use cases (place in subfolder `/docs)`.

4. An **oral presentation** (you need to book a slot – t.b.d.).